22AIE205 — Introduction to Python

Python

Programming Language

Unit 1 · Weeks 1 & 2 | Control Statements · Lists · Tuples · Ranges · Dictionaries · Sets · Input/Output

2nd Year B.Tech CSE | Amrita Vishwa Vidyapeetham, Chennai Campus

Dr. Prabu M

Course Roadmap — What we cover in Weeks 1 & 2

1 Intro to Python

Why Python? Setup, variables, first program

2 Control Statements

if/elif/else · for · while · break · continue

3 Lists & Ranges

Creating, indexing, slicing, list methods

4 Tuples

Immutable sequences, packing & unpacking

5 Dictionaries & Sets

Key-value pairs, set algebra operations

6 Input & Output

input(), print(), f-strings, file read/write

1

Introduction to Python

What is Python? Why learn it? Your first program

What you will learn:

- What is Python?
- Why Python?
- Installing Python
- Variables & Data Types
- First Program
- Comments

What is Python? — And Why Should You Learn It?

🐍 What is Python?

High-level language

You write code in English-like words — Python converts it for the computer.

Interpreted

No separate compilation step. You run code line by line and see results instantly.

General purpose

Used for web apps, AI/ML, data science, automation, games, and more.

Open source & free

Completely free to download, use, and distribute.

⭐ Why Python in 2026?

Easiest to learn first

Python's syntax is clean and minimal — perfect for beginners.

Most popular worldwide

#1 language on TIOBE index. Demanded by every IT company.

Data Science & AI/ML

NumPy, Pandas, TensorFlow, scikit-learn — all Python.

Web Development

Django & Flask power Instagram, Pinterest, YouTube backend.

Automation & Scripting

Automate repetitive tasks — file handling, web scraping, testing.

Variables & Data Types in Python

A variable is a named box that stores a value. In Python, you don't need to declare the type — Python figures it out automatically!

int

```
age = 20
```

Whole numbers (positive, negative, zero). No decimal point.

float

```
gpa = 8.75
```

Decimal numbers. Used for measurements, percentages, fractions.

str

```
name = "Alpha"
```

Text — any characters enclosed in single or double quotes.

bool

```
passed = True
```

Only two values: True or False. Used in conditions.

💡 Python is dynamically typed — the same variable can hold different types at different times.

Your Very First Python Program — Line by Line

Understanding the code:

```
# My first program
```

starts a comment. Python ignores this line. Use comments to explain your code.

```
print("Hello, World!")
```

print() is a built-in function. It displays whatever is inside the brackets on screen.

```
name = "Alpha"
```

We create a variable called name and store the text Alpha in it.

```
age  = 20
```

We create a variable called age and store the number 20 in it.

```
print("Name:", name)
```

print() can take multiple items separated by commas — it adds a space between them.

```
print(f"...{age+5}")
```

f-string: the f before the quote lets you embed variables directly inside {} braces.

hello.py

```
# My first Python program
print("Hello, World!")

name = "Alpha"
age  = 20

print("Name:", name)
print("Age :", age)
print(f"In 5 years, {name} will be {age+5}")
```

OUTPUT

```
Hello, World!
Name: Alpha
Age : 20
In 5 years, Alpha will be 25
```

2

Control Statements

Making decisions and repeating actions in Python

What you will learn:

- if statement
- elif & else
- for loop
- while loop
- break
- continue
- pass
- Nested loops

if / elif / else — Making Decisions in Python

By default Python runs code from top to bottom. The if statement l
condition (something that is True or False).

Syntax (template):

```
if condition1:
    block runs when condition1 is True
elif condition2:
    block runs when condition2 is True
else:
    block runs when ALL above are False
```

3 Golden Rules:

- Always put a colon (:) after the condition.
- The code inside must be indented — 4 spaces or 1 Tab.
- You can have many elif blocks, but only one else.

grade.py

```
marks = 78

if marks >= 90:
    print("Grade: O  (Outstanding)")
elif marks >= 80:
    print("Grade: A+ (Excellent)")
elif marks >= 70:
    print("Grade: A  (Very Good)")
elif marks >= 60:
    print("Grade: B+ (Good)")
else:
    print("Grade: B  (Average)")
```

OUTPUT

```
Grade: A  (Very Good)

(marks=78, so first True condition
is marks>=70 → that block runs)
```

for Loop — Repeating Code for Every Item in a Sequence

A for loop says: 'For each item in this collection, run this block of co
each run.

How it works — step by step:

Step 1 Python looks at the list: ["apple", "banana", "mango"]

Step 2 fruit = "apple" → print(fruit) → prints apple

Step 3 fruit = "banana" → print(fruit) → prints banana

Step 4 fruit = "mango" → print(fruit) → prints mango

Step 5 List is done. Loop ends. Next line runs.

💡 The loop variable (fruit) is created automatically — you choose its name.

for_loop.py

```
# Example 1: loop over a list
fruits = ["apple", "banana", "mango"]
for fruit in fruits:
    print(fruit)

print("Loop finished!")

# Example 2: loop with range
for i in range(1, 6):
    print(i, "x 3 =", i * 3)
```

OUTPUT

```
apple
banana
mango
Loop finished!
1 x 3 = 3
2 x 3 = 6
3 x 3 = 9
4 x 3 = 12
5 x 3 = 15
```

range() — Generating Number Sequences for Loops

range() is a Python built-in that generates a sequence of numbers. It does NOT create a list in memory — it generates one number at a time. Very memory-efficient!

range(stop)

e.g. range(5)

Produces: 0, 1, 2, 3, 4

Always starts from 0. Stop value is NOT included.

```
for i in range(5):
    print(i, end=" ")
# Output: 0 1 2 3 4
```

range(start, stop)

e.g. range(2, 7)

Produces: 2, 3, 4, 5, 6

Start IS included. Stop is NOT included.

```
for i in range(2, 7):
    print(i, end=" ")
# Output: 2 3 4 5 6
```

range(start,stop,step)

e.g. range(0, 10, 2)

Produces: 0, 2, 4, 6, 8

Step controls the jump. Use negative step for countdown.

```
for i in range(0,10,2):
    print(i, end=" ")
# Output: 0 2 4 6 8
```

💡 Countdown: range(10, 0, -1) → 10, 9, 8, ... 1 | list(range(5)) → [0, 1, 2, 3, 4] converts to a list.

while Loop — Repeat as Long as Condition is True

while loop keeps running its block as long as its condition stays True. Use it when you don't know how many times to repeat in advance.

for vs while — When to use which?

for loop

You KNOW how many times to repeat.
Example: process 10 students, loop 5 times.

while loop

You DON'T know how many times.
Example: keep asking until user types 'quit'.

⚠ Infinite Loop Warning:

If the condition NEVER becomes False, the loop runs forever! Always make sure something inside the loop changes the condition — or use break to exit.

while_loop.py

```
# Count from 1 to 5
count = 1
while count <= 5:
    print("Count:", count)
    count = count + 1    # Must change!

print("Loop done!")

# Guess game (simplified)
password = "python"
guess = input("Enter password: ")
while guess != password:
```

OUTPUT

```
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
Loop done!
```

break · continue · pass — Controlling Loop Flow

These three keywords give you fine control over what happens inside a loop.

break

Immediately EXIT the loop — no more iterations.

Use when you've found what you were looking for and don't need to continue.

```
for i in range(1, 10):
    if i == 5:
        break        # stop here
    print(i, end=" ")
print("\nStopped!")
```

```
1 2 3 4
Stopped!
```

continue

SKIP the rest of this iteration, go to the next one.

Use when you want to skip certain values but keep looping.

```
for i in range(1, 8):
    if i % 2 == 0:
        continue    # skip even
    print(i, end=" ")
```

```
1 3 5 7
```

pass

Do NOTHING — placeholder for empty code blocks.

Python requires at least one line in a block. pass fills that gap.

```
for i in range(5):
    pass    # will add code later
print("Done")
```

```
Done
```

💡 Memory trick: break = quit the whole loop | continue = skip this round | pass = I'll code this later

Nested Loops — A Loop Inside Another Loop

You can put a loop inside another loop — this is called nesting. The i
loop. Great for patterns, tables, and 2D data.

How nested loops execute:

- Outer i=1 → Inner j=1,2,3 → prints 1×1, 1×2, 1×3
- Outer i=2 → Inner j=1,2,3 → prints 2×1, 2×2, 2×3
- Outer i=3 → Inner j=1,2,3 → prints 3×1, 3×2, 3×3
- Total executions of inner body = 3 × 3 = 9 times

💡 Outer loops are slow — avoid deeply nested loops on large data.

nested.py

```
# Multiplication table
for i in range(1, 4):
    for j in range(1, 4):
        result = i * j
        print(i,"x",j,"=",result, end="  ")
    print()     # new line after each row

# FizzBuzz (classic interview problem)
for n in range(1, 16):
    if n % 15 == 0: print("FizzBuzz")
    elif n % 3 == 0: print("Fizz")
    elif n % 5 == 0: print("Buzz")
```

OUTPUT

```
1x1=1  1x2=2  1x3=3
2x1=2  2x2=4  2x3=6
3x1=3  3x2=6  3x3=9

1  2  Fizz  4  Buzz  Fizz
7  8  Fizz  Buzz  11  Fizz
13  14  FizzBuzz
```

3

Lists, Ranges & Tuples

Ordered sequences — mutable lists vs immutable tuples

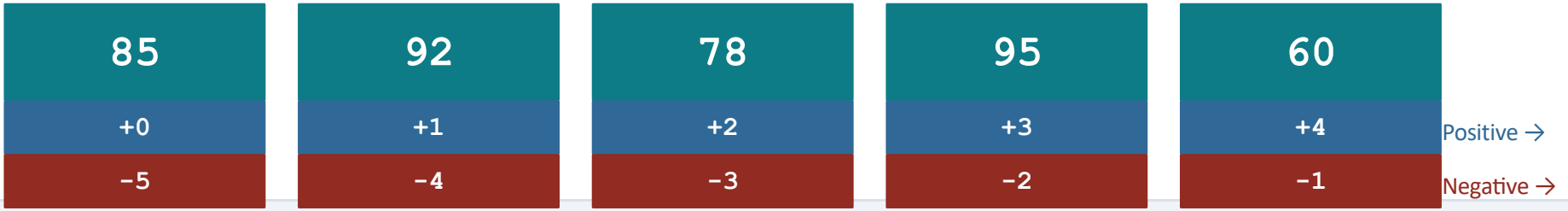
What you will learn:

- Creating lists
- Indexing
- Negative indexing
- Slicing
- List methods
- List comprehension
- Tuples
- Packing & Unpacking

Python Lists — Creating & Accessing Elements

A list is an ordered collection of items. Think of it as a row of numbered boxes — each box holds one value. Lists are MUTABLE, meaning you can change them after creation.

Index diagram — list called 'marks':



list_access.py

```
marks = [85, 92, 78, 95, 60]

print(marks[0])     # 85   (first)
print(marks[2])     # 78   (third)
print(marks[-1])    # 60   (last)
print(marks[-2])    # 95   (second-last)
```

OUTPUT

```
85
78
60
95
```

List Slicing — Extracting a Portion of a List

Slicing lets you grab a chunk of a list. Syntax: list[start : stop : step]. The START is included, the STOP is NOT included. This is like selecting a range on a number line.

`marks[1:4]`

→ `[92, 78, 95]`

Indexes 1, 2, 3 — stop (4) is excluded.

`marks[:3]`

→ `[85, 92, 78]`

No start given → starts from index 0 by default.

`marks[2:]`

→ `[78, 95, 60]`

No stop given → goes to the end of the list.

`marks[::2]`

→ `[85, 78, 60]`

Step = 2 → picks every other item (0, 2, 4).

`marks[::-1]`

→ `[60, 95, 78, 92, 85]`

Step = -1 → reverses the entire list.

`marks[1:4:2]`

→ `[92, 95]`

From index 1 to 3, taking every 2nd item.

💡 Slicing creates a NEW list — the original list is NOT changed. | marks = [85, 92, 78, 95, 60] throughout.

List Methods — Powerful Built-in Operations

Python lists come with built-in methods (actions). Call them with: list_name.method_name(). Here are the most important ones:

`.append(x)`	**Add x to the END of the list.**
	`students.append("Raj")`
	`→ adds 'Raj' at end`

`.insert(i,x)`	**Add x at position i (shifts others right).**
	`students.insert(1, "Meena")`
	`→ at index 1`

`.remove(x)`	**Delete the FIRST occurrence of value x.**
	`students.remove("Ravi")`
	`→ removes 'Ravi'`

`.pop(i)`	**Remove AND return item at index i. Default: last.**
	`students.pop()`
	`→ removes last item`

`.sort()`	**Sort items in ascending order (in-place).**
	`students.sort()`
	`→ alphabetical order`

`.reverse()`	**Reverse the order of items (in-place).**
	`marks.reverse()`
	`→ reverses the list`

`.index(x)`	**Return the index of the first occurrence of x.**
	`marks.index(92)`
	`→ returns 1`

`len(list)`	**Return the total number of items. (Built-in function)**
	`len(students)`
	`→ returns count`

List Methods — Full Code Example

What happens step by step:

`Start`	students = ['Ravi', 'Alpha', 'Arun']
`append('Divya')`	['Ravi', 'Alpha', 'Arun', 'Divya'] — Divya added at end
`insert(1,'Meena')`	['Ravi','Meena','Alpha','Arun','Divya'] — Meena at index 1
`remove('Arun')`	['Ravi','Meena','Alpha','Divya'] — Arun gone
`sort()`	['Divya','Meena','Alpha','Ravi'] — alphabetical
`len()`	4 — there are 4 items in the list

list_methods.py

```
students = ["Ravi", "Alpha", "Arun"]
print("Start:", students)

students.append("Divya")
print("After append:", students)

students.insert(1, "Meena")
print("After insert:", students)

students.remove("Arun")
print("After remove:", students)

students.sort()
print("After sort :", students)
```

OUTPUT

```
Start: ['Ravi', 'Alpha', 'Arun']
After append:
['Ravi','Alpha','Arun','Divya']
After insert:
['Ravi','Meena','Alpha','Arun','Divya']
After remove:
```

List Comprehension — The Pythonic Short-cut

List comprehension is a compact, readable way to create a new list
just ONE line instead of 3-4 lines.

Normal way (4 lines):

```
squares = []
for x in range(1, 6):
    squares.append(x ** 2)
print(squares)   # [1, 4, 9, 16, 25]
```

List comprehension (1 line):

```
squares = [x**2 for x in range(1,6)]
print(squares)   # [1, 4, 9, 16, 25]

# Read as: "x squared for each x in range
1-5"
```

comprehension.py

```
# Squares of 1 to 5
squares = [x**2 for x in range(1, 6)]
print(squares)

# Even numbers only (with condition)
evens = [x for x in range(1,11) if x%2==0]
print(evens)

# Convert names to UPPERCASE
names = ["ravi", "Alpha", "arun"]
upper = [n.upper() for n in names]
print(upper)

# Marks above 80
marks = [85, 92, 68, 95, 55, 78]
high  = [m for m in marks if m >= 80]
print(high)
```

OUTPUT

```
[1, 4, 9, 16, 25]
[2, 4, 6, 8, 10]
['RAVI', 'Alpha', 'ARUN']
```

Python Tuples — Immutable (Read-Only) Sequences

A tuple is exactly like a list — but IMMUTABLE, meaning once create
data that should NOT change (e.g. coordinates, RGB colours, databa

List vs Tuple

Feature	List [ ]	Tuple ( )
Brackets	Square [ ]	Round ( )
Mutable?	YES — can change	NO — read-only
Speed	Slower	Faster
When to use	Data that changes	Fixed/constant data
As dict key?	NO (unhashable)	YES (hashable)

💡 Single-item tuple MUST have a trailing comma: t = (42,) — NOT t = (42)

tuples.py

```
# Creating tuples
point  = (3, 7)
colors = ("red", "green", "blue")
mixed  = (1, "hello", 3.14, True)
single = (42,)    # trailing comma!

# Accessing (same as list)
print(point[0])    # 3
print(colors[-1])  # blue

# Packing
coords = 10, 20    # auto-packed into tuple
print(coords)      # (10, 20)
```

OUTPUT

```
3
blue
(10, 20)
x=3, y=7
red green blue
```

4

Dictionaries & Sets

Key-value storage and unique collections

What you will learn:

- Creating dicts
- Accessing values
- Adding & updating
- Deleting
- Dict methods
- Creating sets
- Set operations
- Frozenset

Python Dictionaries — Storing Data as Key : Value Pairs

A dictionary stores data as key:value pairs — like a real dictionary (w
position number. Keys must be unique.

Key rules about dictionaries:

Keys	Must be UNIQUE. Duplicate keys overwrite the earlier value.
Keys	Must be IMMUTABLE: strings, numbers, tuples are valid keys.
Values	Can be ANYTHING: numbers, strings, lists, even other dicts!
Order	Python 3.7+ preserves insertion order.
Access	Use dict[key] to get the value. If key missing → KeyError.
Safe access	Use dict.get(key) to get None instead of error.

💡 Analogy: Phone book — Name (key) → Phone number (value).

dict_basics.py

```
# Creating a dictionary
student = {
    "name" : "Alpha",
    "roll" : 101,
    "marks": [85, 90, 78]
}

# Accessing values
print(student["name"])          # Alpha
print(student.get("age", "N/A")) # N/A
(safe)

# Add a new key
student["age"] = 20
print(student["age"])           # 20
```

OUTPUT

```
Alpha
N/A
20
{'name':'Alpha','marks':[85,90,78],'age':20}
```

Dictionary Methods & Looping Through a Dictionary

Essential Dictionary Methods:

`.keys()`	Returns all KEYS as a view object
`.values()`	Returns all VALUES as a view object
`.items()`	Returns all (key, value) PAIRS
`.get(k, default)`	Returns value for k; default if missing
`.update(dict2)`	Merges dict2 into this dictionary
`.pop(key)`	Removes key and returns its value
`.setdefault(k,v)`	Sets key to v only if key is absent
`len(dict)`	Returns the number of key-value pairs
`key in dict`	True if key exists, False otherwise

dict_loop.py

```
scores = {
    "Ravi" : 85,
    "Alpha": 92,
    "Arun" : 78
}

# Loop through keys
for name in scores.keys():
    print(name)

# Loop through key-value pairs
for name, mark in scores.items():
    print(f"{name}: {mark}")
```

OUTPUT

```
Ravi
Alpha
Arun
Ravi: 85
Alpha: 92
Arun: 78
```

Python Sets — Collections of Unique Items

A set is an UNORDERED collection with NO DUPLICATE values. Think
perfect for removing duplicates and doing mathematical set operati

Important properties of Sets:

No duplicates	If you add the same item twice, it still appears only once.
Unordered	Items have no fixed position — you cannot use indexing.
Mutable	You can add/remove items after creation.
Heterogeneous	Can hold different types — but no lists (unhashable).
Empty set	Write set() — NOT {} (that creates an empty dict!).

💡 Most common use: remove duplicates from a list → unique = list(set(my_list))

sets_intro.py

```
# Creating a set
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

# Auto-removes duplicates
numbers = {1, 2, 2, 3, 3, 3, 4}
print(numbers)     # {1, 2, 3, 4}

# Remove duplicates from a list
marks = [85, 90, 85, 78, 90, 100]
unique_marks = list(set(marks))
```

OUTPUT

```
{1, 2, 3, 4}
[100, 78, 85, 90]  (order may vary)
```

Set operations: A={1,2,3,4,5} B={4,5,6,7,8}

A|B Union → 1-8	A&B Inter. → 4,5
A-B Diff → 1,2,3	A^B Sym → 1,2,3,6,7,8

Set Methods & Operations — Full Code

Set Methods:

`.add(x)`	Add element x to the set
`.update(s)`	Add all elements from s
`.remove(x)`	Remove x (KeyError if missing)
`.discard(x)`	Remove x (NO error if missing)
`.pop()`	Remove and return a random element
`.clear()`	Remove all elements
`.issubset(s)`	True if all items of self are in s
`.issuperset(s)`	True if self contains all of s
`.isdisjoint(s)`	True if no common elements
`frozenset(s)`	Create immutable (unchangeable) set

set_methods.py

```
batch_A = {"Ravi", "Alpha", "Arun"}
batch_B = {"Alpha", "Arun", "Meena"}

# Add & discard
batch_A.add("Divya")
print(batch_A)
batch_A.discard("Alpha")
print(batch_A)

# Set operations
print("Common:",   batch_A & batch_B)
print("All:",      batch_A | batch_B)
print("Only A:",   batch_A - batch_B)
```

OUTPUT

```
{'Ravi', 'Alpha', 'Arun', 'Divya'}
{'Ravi', 'Arun', 'Divya'}
Common: {'Arun'}
All: {'Ravi','Alpha','Arun','Meena','Divya'}
Only A: {'Ravi', 'Divya'}
True
```

5

Input & Output in Python

Talking to the user — reading input, displaying output, file operations

What you will learn:

- input() basics
- Type conversion
- print() options
- sep & end
- f-strings
- format()
- File write
- File read

input() — Reading Data from the User

input() pauses the program and waits for the user to type something
types a number. You must convert the type yourself.

Why type conversion is needed:

```
age = input("Enter age: ")  # user types 20
print(type(age))  # <class 'str'> ← it's a STRING!

# This will CRASH:
# print(age + 5)  → TypeError!

# Correct:
age = int(input("Enter age: "))
print(age + 5)  # works fine → 25
```

Type conversion functions:

`int(x)`	Convert to integer whole number → int("20") → 20
`float(x)`	Convert to decimal number → float("3.14") → 3.14
`str(x)`	Convert to string (text) → str(42) → "42"

input_demo.py

```
# Basic input
name = input("Enter your name: ")
print("Hello,", name + "!")

# Integer input
age = int(input("Enter your age: "))
print("Next year you will be", age + 1)

# Float input
height = float(input("Height in cm: "))
bmi_h  = height / 100
print(f"Height in metres: {bmi_h:.2f}")

# Multiple inputs on one line
```

OUTPUT

```
Enter your name: Alpha
Hello, Alpha!
Enter your age: 20
Next year you will be 21
Height in cm: 165
Height in metres: 1.65
Enter two numbers: 10 20
```

print() & String Formatting — Displaying Output Beautifully

print() is your main tool for showing output. Beyond basic use, Pyth
output messages. The modern and best way is f-strings.

print() extra parameters:

sep = Separator placed BETWEEN the values you print (default: single space).

```
print(1,2,3, sep="-") → 1-2-3
```

end = What to print at the END of the line (default: newline \n).

```
print("Hi", end="") → stays on same line
```

3 Ways to format strings:

% operator (old) `"Hello %s, age %d" % (name, age)`

formatting.py

```
name  = "Ravi"
marks = 92.5
rank  = 1

# sep and end
print(1, 2, 3, sep=" | ")
print("Hi", end=" ")
print("there")          # same line!

# f-string (recommended)
print(f"Student : {name}")
print(f"Marks   : {marks:.1f}")    # 1
decimal
```

OUTPUT

```
1 | 2 | 3
Hi there
Student : Ravi
Marks   : 92.5
Rank    : 1
Passed? : True
```

File Input & Output — Reading & Writing Files

Python can read from and write to files on disk. Always use 'with op
closes the file when done, even if an error occurs.

File modes explained:

'r' — **READ**

Open for reading (default). File must already exist.

'w' — **WRITE**

Open for writing. Creates file if not found. OVERWRITES existing content!

'a' — **APPEND**

Open for writing. Creates file if not found. ADDS to the end.

'x' — **CREATE**

Create a new file. Raises error if file already exists.

file_io.py

```
# --- WRITE to a file ---
with open("marks.txt", "w") as f:
    f.write("Ravi   85\n")
    f.write("Alpha  92\n")
    f.write("Arun   78\n")
print("File written!")

# --- READ the whole file ---
with open("marks.txt", "r") as f:
    content = f.read()
print(content)
```

OUTPUT

```
File written!
Ravi   85
Alpha  92
Arun   78

Ravi   85
Alpha  92
Arun   78
```

Weeks 1 & 2 — Complete Summary

Variables & Types	int, float, str, bool · dynamic typing · no declaration needed	**Control Statements**	if/elif/else · for · while · break · continue · pass · nested
Lists	Ordered, mutable · index/slice · 8+ methods · comprehension	**range()**	range(stop) · range(start,stop) · range(start,stop,step)
Tuples	Immutable · packing · unpacking · faster · hashable	**Dictionaries**	key:value · .keys() .values() .items() · dict comprehension
Sets	Unique · unordered · union | intersect & difference - sym ^	**Input / Output**	input() + type-convert · print(sep,end) · f-strings · file I/O

Reference: Allen B. Downey, Think Python, 2nd Ed., O'Reilly, 2016 | Practice: Python IDLE · VS Code · Jupyter Notebook